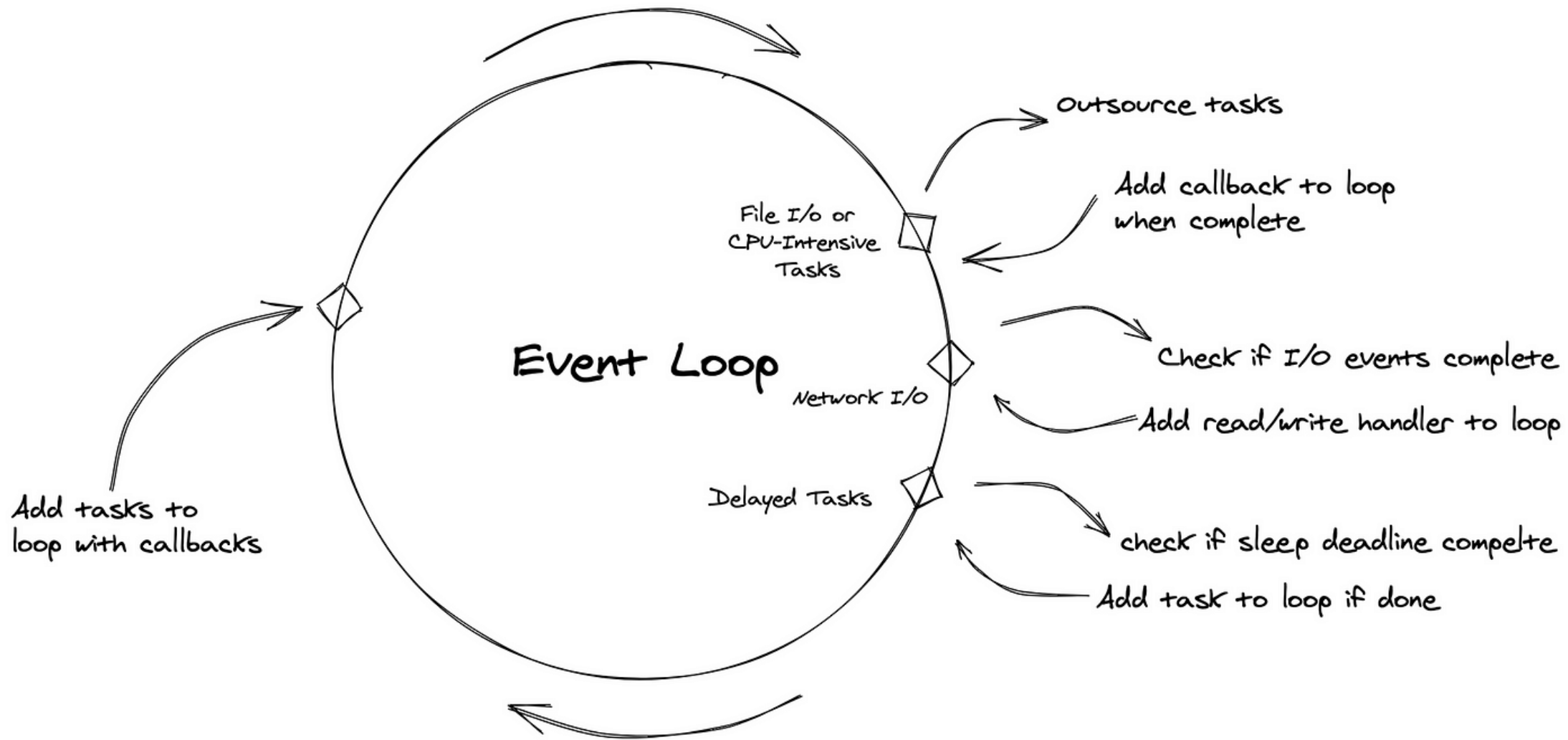


Asynchronous Task



Welcome to Asyncio Tasks (Advanced)

- What is an Asyncio Task?
 - A **wrapper** for a **coroutine** that **schedules** it to run on the **Event Loop**.
 - Subclass of **asyncio.Future**, inheriting all future-like capabilities.

Topics Today

- Task **Inspection & Lifecycle** (Done, Canceled, Results, Exceptions)
- Task **Manipulation** (Cancellation, Callbacks, Naming)
- Global Task **Management** (current_task, all_tasks)
- Grouping & **Waiting** Strategies (gather vs wait, Timeouts)

What is an Asyncio Task

- A wrapper for a coroutine that schedules it to run on the Event Loop.
- Subclass of `asyncio.Future`, inheriting all future-like capabilities.

How to Create a Task

- Create Task With High-Level API (preferred)
- Create Task With Low-Level API

Create Task with High-Level API

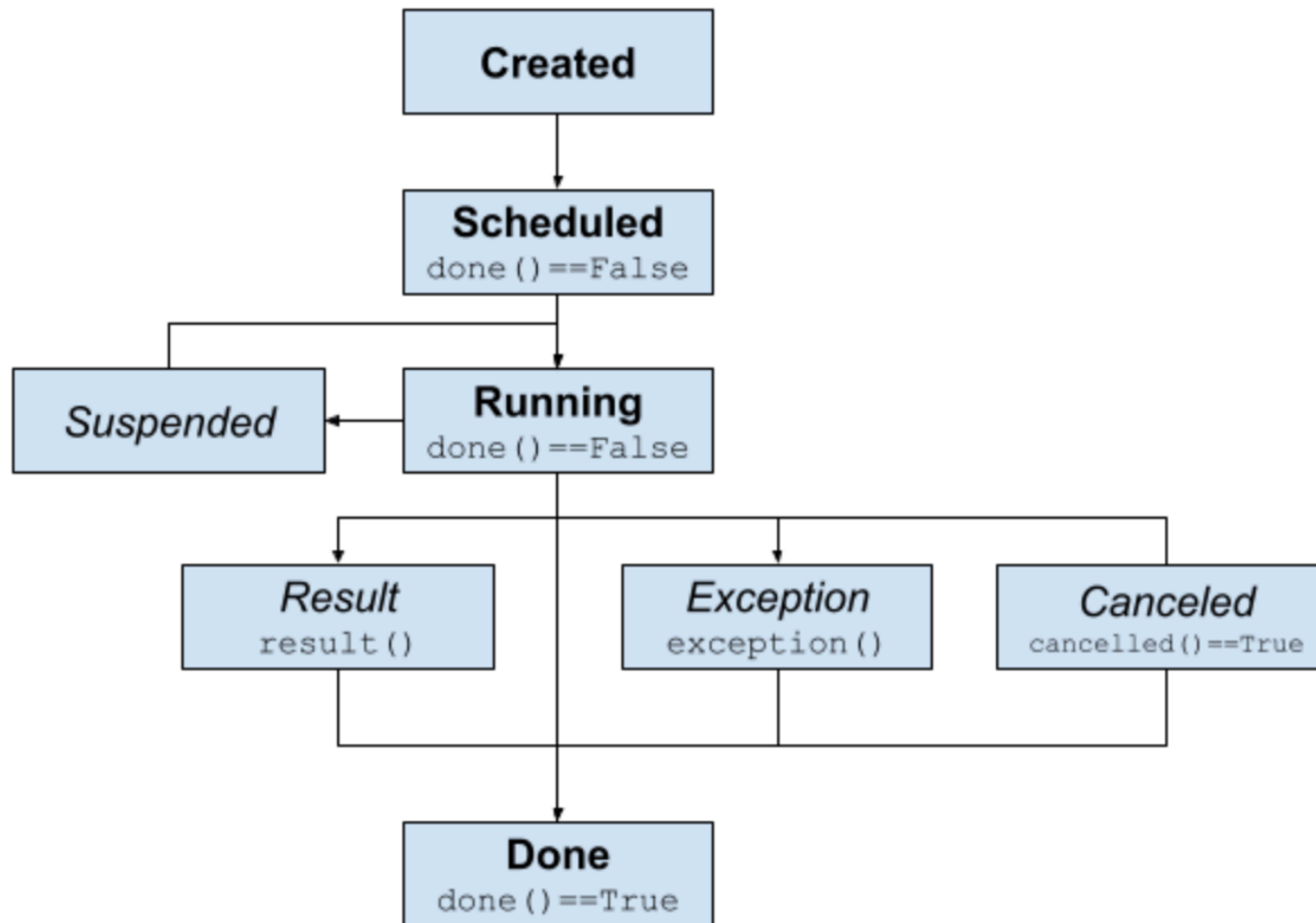
- The **`asyncio.create_task()`** function takes a coroutine instance and an optional name for the task and returns an `asyncio.Task` instance
 - Wrap the coroutine in a Task instance.
 - Schedule the task for execution in the current event loop.
 - Return a Task instance

```
1 ...  
2 # create a coroutine  
3 coro = task_coroutine()  
4 # create a task from a coroutine  
5 task = asyncio.create_task(coro)
```

```
1 ...  
2 # create a task from a coroutine  
3 task = asyncio.create_task(task_coroutine())
```


Create Task With High-Level API is the preferred way to create a Task from a coroutine in an asyncio program.

Asyncio Task Life-Cycle



Check if a Task is Done

- `task.done()`
 - Returns **True** if the task has **finished** executing.
 - A task is "**done**" if it **returned** a value, **raised** an exception, or was **canceled**.

```
task = asyncio.create_task(some_coroutine())
# ... later in the loop ...
if task.done():
    print("Task has finished its execution.")
if task.cancelled():
    print("Task was stopped prematurely.")
```

Check if a Task is Cancelled

- `task.cancelled()`
 - Returns **True** if the task was successfully **canceled before normal completion**

```
task = asyncio.create_task(some_coroutine())
# ... later in the loop ...
if task.done():
    print("Task has finished its execution.")
if task.cancelled():
    print("Task was stopped prematurely.")
```

How to Get Task Result

result()

- `task.result()`
 - Returns the **value** returned by the coroutine.
 - Warning: Raises **InvalidStateError** if the task isn't done yet. If the task raised an exception, calling **result()** re-raises that exception.

```
try:
    value = task.result()
except asyncio.CancelledError:
    print("No result, task was canceled.")
except Exception as e:
    print(f"Task crashed with: {task.exception()}")
```

How to Get Task Result

result()

- `task.exception()`
 - Returns the **exception object** raised by the coroutine.
 - Returns **None** if the task finished successfully without errors.

```
try:
    value = task.result()
except asyncio.CancelledError:
    print("No result, task was canceled.")
except Exception as e:
    print(f"Task crashed with: {task.exception()}")
```

How to Cancel a Task

- `task.cancel()`
 - **Requests** the task to **stop execution**.
 - Injects an **`asyncio.CancelledError`** exception into the coroutine at its **next await point**.
 - The coroutine can **catch** this **exception** to clean up **resources** (e.g., closing database connections).

How to Cancel a Task

- ```
async def worker():
 try:
 await asyncio.sleep(10)
 except asyncio.CancelledError:
 print("Cleaning up resources before exiting...")
 raise # Re-raise to finalize cancellation

task = asyncio.create_task(worker())
await asyncio.sleep(1)
task.cancel() # Schedulers cancellation
```



# How to Use Callback With a Task

- `task.add_done_callback(callback_function)`
  - Attaches a **regular function (not a coroutine)** to be executed immediately when the task becomes "**done**".
  - The task object itself is automatically passed as the first argument to the callback.

```
def on_complete(future_task):
 print(f"Notification: Task finished! Result = {future_task.result()}")

task = asyncio.create_task(calculate_data())
task.add_done_callback(on_complete) # Register callback
```

# How to Set the Task Name

- `task.set_name(name)` & `task.get_name()`
  - Allows you to **assign** a **human-readable string** name to a specific task.
  - Extremely useful for **debugging**, **logging**, and **tracking** dynamic background activities.

```
task = asyncio.create_task(fetch_api(), name="API-Fetcher-Core")
print(f"Task running: {task.get_name()}")
```

```
Output: Task running: API-Fetcher-Core
```

# Current and Running Tasks

# How to Get the Current Task

- `asyncio.current_task()`
  - Returns the **current** Task instance executing inside the active coroutine.

```
Inside a coroutine
current = asyncio.current_task()
print(f"I am executing inside: {current.get_name()}")
```

# How to Get All Tasks

- `asyncio.all_tasks()`
  - Returns a Python set containing **all currently running (uncompleted)** tasks managed by the active Event Loop.

```
Check total active workload
running_tasks = asyncio.all_tasks()
print(f"Active tasks in event loop: {len(running_tasks)}")
```

**Run Many Coroutines  
Concurrently**

# What is Asyncio gather()

- `asyncio.gather(*aws)`
  - Runs multiple awaitable objects (coroutines/tasks) **concurrently**.
  - **Key Feature:** Returns a structured **list of results** matching the **exact order** of the inputs after ALL items complete.
  - If any task fails, you can control the error behavior using `return_exceptions=True/False`.

**Wait for A Collection of Tasks**



# What is `asyncio.wait()`

- `asyncio.wait(fs)`
  - **Monitors** a collection of Task objects and returns when specific conditions are met.
  - **Returns two sets** of tasks: (**done**, **pending**).
- Flexible Control with `return_when`:
  - **ALL\_COMPLETED** (Default)
  - **FIRST\_COMPLETED** (Returns immediately when the first task finishes)
  - **FIRST\_EXCEPTION** (Returns if any task crashes)

# What is `asyncio.wait()`

- `asyncio.wait(fs)`

```
tasks = {asyncio.create_task(worker(i)) for i in range(3)}
Return as soon as the speediest task is done
done, pending = await asyncio.wait(tasks, return_when=asyncio.FIRST_COMPLETED)
```

- Flexible Control with `return_when`:

# Wait for a Coroutine with a Time Limit

- `asyncio.wait_for(aw, timeout)`
  - Sets a strict **maximum time** boundary for an operation to finish.
  - If the timeout **expires**, the target task is automatically canceled, and `TimeoutError` is raised.

```
try:
 # If fetch_heavy_report takes > 2 seconds, cut it off!
 result = await asyncio.wait_for(fetch_heavy_report(), timeout=2.0)
except asyncio.TimeoutError:
 print("The server took too long to respond. Request aborted.")
```

# asyncio.gather() vs asyncio.wait()

| Feature          | <code>asyncio.gather()</code>                      | <code>asyncio.wait()</code>                          |
|------------------|----------------------------------------------------|------------------------------------------------------|
| Primary Input    | List of Coroutines / Awaitables                    | Set of initialized <b>Task</b> objects               |
| Return Value     | Ordered <code>list</code> of task returned results | Tuple of two task sets: <code>(done, pending)</code> |
| Short-circuiting | Cannot stop early on first success                 | Can stop early via <code>FIRST_COMPLETED</code>      |
| Best Used For    | Aggregating data, fetching unified responses       | Complex workflows, pipelines, racing tasks           |

# Exercise

# Assignment

- **Assignment 1:** Smart Courier System (40 คะแนน) -> ตรวจสอบการคุมและการจัดการ CanceledError
- **Assignment 2:** Stock Price Race (Mock Version) (50 คะแนน) -> ตรวจสอบลอจิก FIRST\_COMPLETED ผ่าน asyncio.sleep
- **Assignment 3:** Stock Price Race (Live FastAPI + HTTPX) (10 คะแนน) -> ประยุกต์ใช้ Concurrency บนระบบเครือข่ายจำลองจริงร่วมกับ httpx

# Assignment 1: The Smart Courier System (ระบบส่งพัสดุด่วน)

- โจทย์: ให้นักศึกษาจำลองระบบพนักงานส่งพัสดุ 1 คนที่ต้องไปส่งของให้ลูกค้าแบบออนไลน์ โดยพนักงานสามารถตรวจเช็คสถานะงานและยกเลิกงานได้หากใช้เวลานานเกินไป
- ข้อกำหนดในการเขียนโค้ด:
  1. สร้าง Coroutine function ชื่อ **delivery\_task(package\_id, duration)** ให้จำลองการส่งของด้วย **asyncio.sleep(duration)** และให้พิมพ์ข้อความเมื่อเริ่มส่ง และเมื่อส่งเสร็จสิ้นให้ **return f"Package {package\_id} Delivered!"**
  2. ภายในฟังก์ชัน **main()** ให้สร้าง Task จาก **delivery\_task** ขึ้นมา 1 ตัว โดยส่งค่า **package\_id="P001"** และ **duration=5.0** (ใช้เวลาส่ง 5 วินาที) และทำการตั้งชื่อ Task นี้ว่า **"Express-Courier"**
  3. ให้เขียนโปรแกรมจำลองว่าระหว่างที่พัสดุกำลังเดินทาง (เช่น ผ่านไป 2 วินาที) ให้ทำการตรวจสอบด้วยคำสั่งแฝงว่า Task นี้เสร็จหรือยัง (**.done()**) และสั่งพิมพ์ชื่อของ Task ปัจจุบันออกมาดูบนหน้าจอ
  4. หากพบว่าส่งของนานเกินไป (ผ่านไป 2 วินาทีแล้วยังไม่เสร็จ) ให้โปรแกรมหลักทำการยกเลิกงานนั้นทันทีด้วย **.cancel()**
  5. ดักจับข้อผิดพลาด **asyncio.CancelledError** ในตัวครูทีนส่งของ เพื่อพิมพ์ข้อความระบุว่า **"Delivery Canceled! Returning package to warehouse."** และตรวจสอบสถานะตัวแปรภายนอกว่า **.cancelled()** เป็น True หรือไม่

## Assignment 2: The Stock Price Race (ระบบแข่งดึงข้อมูลราคาหุ้น)

- **โจทย์:** ระบบจำเป็นต้องดึงข้อมูลราคาหุ้นจากเซิร์ฟเวอร์ 3 สาขา (Alpha, Beta, Gamma) ซึ่งแต่ละสาขามีความเร็วในการตอบสนองอินเทอร์เน็ตไม่เท่ากัน เราต้องการผลลัพธ์จาก สาขาที่ไวที่สุดเพียงสาขาเดียว มาใช้งานทันทีเพื่อความรวดเร็ว และให้ยกเลิกงานของสาขาที่เหลือทิ้งเพื่อประหยัดทรัพยากรระบบ
- ข้อกำหนดในการเขียนโค้ด:
  - สร้าง Coroutine function ชื่อ `fetch_stock_price(server_name, delay)` ภายใน ให้ `asyncio.sleep(delay)` และส่งค่ากลับเป็นข้อความราคาหุ้น เช่น `return f"[{server_name}] Price: 150 USD"`
  - ภายในฟังก์ชัน `main()` ให้แตก Task ขึ้นมา 3 ตัวพร้อมกันใน Event Loop โดยส่งข้อมูลดังนี้:
    - เซิร์ฟเวอร์ Alpha (delay = 3.0 วินาที)
    - เซิร์ฟเวอร์ Beta (delay = 0.8 วินาที)
    - เซิร์ฟเวอร์ Gamma (delay = 1.5 วินาที)
  - ให้นักศึกษาเลือกใช้กลไกกลุ่มฟังก์ชันที่เหมาะสมระหว่าง `asyncio.gather()` หรือ `asyncio.wait()` เพื่อให้ระบบตัดตัวหลุดจากการรอทันทีเมื่อมีเซิร์ฟเวอร์ตัวแรกส่งข้อมูลกลับมาสำเร็จ (FIRST\_COMPLETED)
  - แสดงผลลัพธ์ของเซิร์ฟเวอร์ที่ชนะการแข่งขัน (ตัวที่เร็วที่สุด) ออกทางหน้าจอ
  - เขียนคำสั่งวงลูปเคลียร์ระบบอัตโนมัติ เพื่อสั่งยกเลิก (`.cancel()`) งานของเซิร์ฟเวอร์อีก 2 ตัวที่เหลือที่ยังทำงานค้างอยู่ (pending) ให้หมดสิ้น เพื่อป้องกันปัญหา Memory Leak หลังบ้าน



# Assignment 3: Concurrency Racing on Live Network

- ระบบของนักศึกษาจะต้องเชื่อมต่อกับ Stock Price API Server ของอาจารย์ (ที่รันด้วย FastAPI บนพอร์ต 8088) ซึ่งแต่ละ Endpoint สาขาจะถูกจำลอง อินเทอร์เน็ต (Latency) ไว้ไม่เท่ากันดังนี้:
  - Alpha -> อินเทอร์เน็ตช้ามาก (หน่วงเวลา 3.0 วินาที)
  - Beta -> อินเทอร์เน็ตเร็วที่สุด (หน่วงเวลา 0.8 วินาที) [คาดหวังเป็นผู้ชนะ]
  - Gamma -> อินเทอร์เน็ตปานกลาง (หน่วงเวลา 1.5 วินาที)
-

# Assignment 3: Concurrency Racing on Live Network

- ให้นักศึกษาสร้างไฟล์ชื่อ `stock_price_httpx.py` และเขียนโปรแกรมตามเงื่อนไขต่อไปนี้:
- ฟังก์ชัน `fetch_stock_price(server_name: str)`:
  - ต้องประกาศเป็น Coroutine function (`async def`)
  - ห้ามรับพารามิเตอร์ `delay` เด็ดขาด (เนื่องจากความหน่วงจะเกิดขึ้นจริงที่ฝั่ง API Server ของอาจารย์)
  - สร้างโครงสร้างตรรกะเชื่อมต่อไปยัง URL: `http://172.16.2.117:8088/price/{server_name}`
  - ต้องใช้งาน `httpx.AsyncClient()` ผ่านบล็อกโครงสร้าง `async with` เพื่อดึงข้อมูล JSON มาจัดฟอร์แมตแสดงผลในรูปแบบสตริง เช่น "[Beta] Price: 149.8 USD"
- ฟังก์ชัน `main()` และลอจิก Concurrency Racing:
  - ทำการแปลงฟังก์ชันคอร์รูทีนดึงข้อมูลของทั้ง 3 สาขา (Alpha, Beta, Gamma) ให้กลายเป็น `asyncio.Task` ด้วยคำสั่ง `asyncio.create_task()` เพื่อส่งเข้าคิวรันพร้อมกันใน Event Loop
  - ใช้คำสั่ง `asyncio.wait()` และระบุ 옵션เงื่อนไข `return_when=asyncio.FIRST_COMPLETED` เพื่อให้โปรแกรมหลักดีดตัวหลุดจากการรอทันทีเมื่อมีเซิร์ฟเวอร์ตัวแรกส่งข้อมูลกลับมาสำเร็จ
  - ดึงผลลัพธ์จากเซิร์ฟเวอร์ที่ชนะการแข่งขัน (ตัวที่เร็วที่สุด) ออกมาพิมพ์แสดงบนหน้าจอ
  - [สำคัญมาก - Anti-Memory Leak]: เขียนคำสั่งวนลูปดึงงานที่ยังคงค้างคาอยู่ในเซต `pending` มาสั่งยกเลิกงานทิ้งให้หมดสิ้นด้วยคำสั่ง `.cancel()` เพื่อตัดสัญญาณ Network Request ที่ยังวิ่งค้างอยู่บนระบบเครือข่าย